

MovAni

Technical Architecture & System Report

An End-to-End Analysis of the MovAni Web Platform

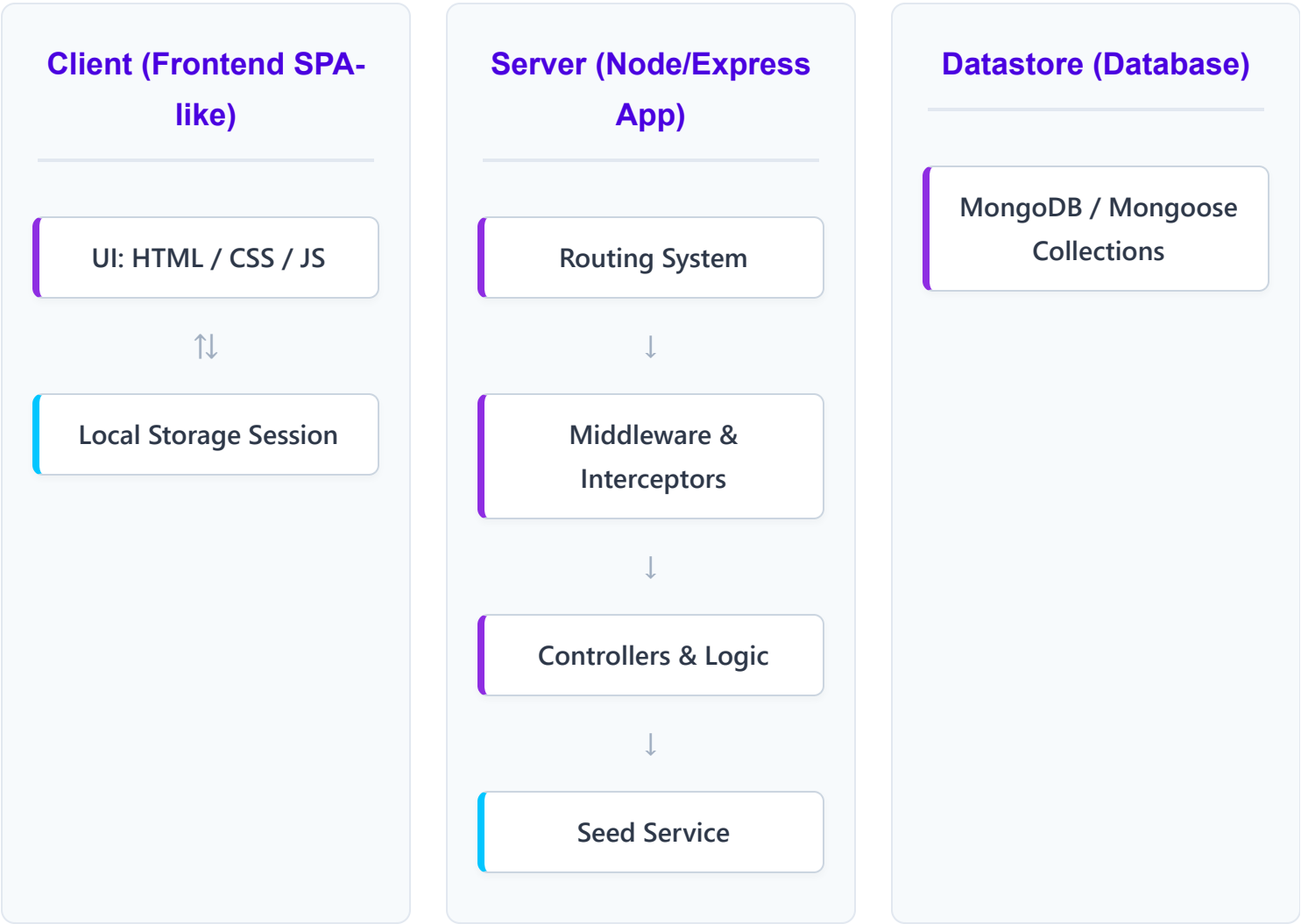
Prepared by: Antigravity AI

Date: July 26, 2026

This report provides an in-depth, end-to-end technical overview of **MovAni (Movie & Anime Reviews Hub)**. It details the system architecture, backend REST APIs, database design, frontend styling, client-side functionality, and key application workflows.

1. System Architecture Overview

MovAni is built using a hybrid decoupled/monolithic structure. The project structure consists of a **Node.js/Express.js backend** and a **static HTML/CSS/JS frontend**.



Integration Strategy

- **Static File Hosting:** The Express server hosts the frontend directory static files directly via `express.static(...)`.
- **Single Page Application Fallback:** The server routes all unmatched routes (`*`) to `frontend/index.html` to enable clean client-side url management.
- **Consolidated Data Route:** To bridge data seamlessly, the server provides a dedicated `/data.json` aggregated route. This endpoint query-joins all models (Movies, Anime, Reviews, Users, Watchlists) into one fast JSON response, minimizing roundtrips during frontend initialization.

2. Backend Architecture

The backend code is organized into a modular **Model-View-Controller (MVC)** directory structure:

```
backend/
├─ backups/           # Auto-generated database JSON backups
├─ src/
│   ├─ app.js         # Express app middleware & routes aggregation
│   ├─ server.js       # DB connection & server initialization entry
│   ├─ config/
│   │   └─ db.js       # MongoDB Connection configuration
│   ├─ controllers/    # Controllers for handling business logic
│   ├─ middleware/     # Request interceptors
│   ├─ models/         # Mongoose database models
│   ├─ routes/         # API endpoint definitions
│   └─ services/
│       └─ seedService.js # Database seeding scripts
├─ db.json            # Seed dataset source
├─ package.json
└─ .env
```

2.1 Mongoose Schemas & Data Models

MovAni leverages **MongoDB** via Mongoose models. Key models include:

1. User (`User.js`)

- Fields: `id` (numeric key), `username`, `email`, `password` (bcrypt hashed), `displayName`, `avatar` (initials), `joined` (Date), `role` (`'user'` or `'admin'`), `banned` (Boolean), `stats` ({ `reviews`, `watchlist`, `following`, `followers` }), `preferences` ({ `genres`, `content` }).

2. Movie & Anime (`Movie.js` , `Anime.js`)

- Fields: `id` (numeric key), `title`, `type` (`'movie'` / `'anime'`), `year`, `rating`, `poster`, `banner`, `runtime` / `episodes`, `genres` (Array), `synopsis`, `status` (`'Released'` / `'Completed'` / `'Ongoing'`), `trailer` (YouTube embed URL), `director` / `studio`, `cast` (Array).

3. Review (`Review.js`)

- Fields: `id`, `movieId` (references Movie/Anime numeric ID), `userId` (references User numeric ID), `user` (display name), `avatar`, `rating` (1–10), `title`, `content`, `email`, `metrics` ({ `overall`, `story`, `characters`, `visuals`, `sound` }), `spoiler` (Boolean), `privacy` (`'public'` / `'private'`), `likes` (Number), `comments` (Subdocument Array: { `id`, `userId`, `user`, `avatar`, `content`, `date` }).

4. Watchlist (`Watchlist.js`)

- Fields: `id`, `userId`, `itemId`, `itemType` (`'movie'` / `'anime'`), `status` (`'unwatched'` / `'watching'` / `'completed'` / `'dropped'`), `added` (Date).

5. System Settings (`SystemSettings.js`)

- Flat key-value collection configuration: `key` (String), `value` (Mixed). Used to toggle maintenance mode, site title, blocked IPs, home layouts, and genre categories.

6. Supportive Admin Models

- `Ad.js` : Banner assets, clicks, active status.
- `Report.js` : User moderation complaints on reviews.
- `Notification.js` : System-wide notification alerts.
- `ActivityLog.js` : Database logs of admin activities.

3. Core API Routes & Endpoints

Method	Endpoint	Description	Middleware / Auth
POST	/api/auth/register	Register a new user	Validation check
POST	/api/auth/login	Authenticate credentials & return User	Validation check
POST	/api/auth/logout	Terminate session	None
GET	/api/auth/me	Fetch active user information	x-user-id header
GET	/api/movies	Get filtered list of movies	Genre/Year query parameters
GET	/api/movies/:id	Fetch specific movie by ID	None
GET	/api/anime	Get list of anime titles	Genre/Year query parameters
GET	/api/reviews	Retrieve all reviews (sorted by newest)	User/Movie query parameters
POST	/api/reviews	Submit content review	Logged-in check
POST	/api/reviews/:id/like	Upvote a review card	None
POST	/api/reviews/:id/comments	Add a nested comment thread	None
GET	/api/watchlist	Fetch watchlist items	userId query parameter
POST	/api/watchlist	Add a title to watchlist	Logged-in check

PUT	/api/watchlist/:id	Update watching status	None
DELETE	/api/watchlist	Remove item from watchlist	None
GET	/api/search	Full-text query match on Titles/Genres/Studios	RegEx match

Administrative Endpoints (/api/admin/*)

All admin endpoints are globally protected by `adminAuth.js`.

- **GET** `/api/admin/stats` : Aggregated collection statistics (user growth curves, comments count, pending reports).
- **PUT** `/api/admin/users/:id` : Revoke permissions, promote to admin, or ban accounts.
- **POST/PUT/DELETE** `/api/admin/movies` & `/api/admin/anime` : Direct collection updates.
- **POST/DELETE** `/api/admin/genres` : Manage central tags.
- **POST/DELETE** `/api/admin/backups` : Trigger system state database dumps.

4. Frontend Architecture & Design System

The frontend layout relies heavily on semantic HTML5 structures, smooth interactive JavaScript components, and custom CSS styling.

4.1 CSS Design System (`style.css` , `forms.css` , `details.css` , `admin.css`)

Style systems utilize vanilla CSS custom variables supporting a dark mode theme:

```
:root {
  --primary: #4a00e0;      /* Deep Violet Accent */
  --secondary: #8e2de2;    /* Vibrant Purple */
  --accent: #00c6ff;        /* High contrast neon cyan */
  --dark: #121212;          /* Layout Background */
```

```
--dark-alt: #1e1e1e;      /* Cards & Modals Background */
--light: #ffffff;
--gray: #2d2d2d;
--text: #f5f5f5;
--text-secondary: #aaa;
--rating-high: #00ff88;   /* Green badge */
--rating-medium: #ffaa00; /* Yellow badge */
--rating-low: #ff4757;    /* Red badge */
--movie: #3498db;         /* Movie Badge Color */
--anime: #e74c3c;         /* Anime Badge Color */
}
```

4.2 Key Pages & Features

1. Landing Hub (`index.html` , `script.js`)

- **Hero Slider:** Auto-cycling banner slideshow fetching items from `homepageConfig` with interactive controls.
- **Filter grids:** Split filtering of Movies and Anime by rating, release dates, and category.
- **Interactive Reviews:** Direct feeds of comments, upvote counters, and inline response textareas.

2. Review Form (`add-review.html` , `add-review.js`)

- **Autocomplete Engine:** Listens to query events against `/data.json` to enable instant title lookup with preview poster icons.
- **Sub-metric Rating sliders:** Individual sliders for Story, Visuals, Characters, Sound, and Overall rating.

3. User Authentication (`login.html` , `signup.html` , `auth.js`)

- **Wizard Layout:** Step-by-step registration flow (User details -> Genre preferences -> Verification steps).
- **Demo Mode:** One-click bypass authentication (`demo@movani.com` / `demo123`) that instantly logs in testing users and updates user sessions.

4. Watchlist Dashboard (`watchlist.html` , `watchlist.js`)

- Dynamic calculations of watched statistics (Total Hours, Average rating, plan states).
- Multi-criteria sorting (Added time, Rating index, Alphabetical titles).

5. Details Hub (`details.html` , `details.js`)

- Dynamic layout showing high-res headers, trailers, cast lists, synopses, and nested review sections.

5. Critical System Workflows

5.1 Authentication Workflow

```
[Client App] ---> Check localStorage['currentUser']  
  |---> Present: Display Username & Logout link  
  |---> Absent: Display "Login / Register" buttons  
  
[API Request] ---> Attach 'x-user-id' header with user numeric ID  
  |---> Server decrypts/checks user profile from database  
  |---> Checks ban state or admin role
```

5.2 Maintenance Mode Middleware

The maintenance interceptor handles system outages by checking `SystemSettings` collections:

- If active, it inspects incoming requests:
- Excludes essential login routes (`/api/auth`) and files (`admin.*` , `maintenance.html`) so administrators can log in to disable maintenance mode.
- Allows authenticated administrators to pass through.
- Blocks normal user API requests with a `503 Service Unavailable` response.
- Redirects frontend page navigation requests directly to the `maintenance.html` splash screen.

5.3 Automated Backups System

Admins can trigger system backups directly:

1. Gathers all records in MongoDB (Movies, Anime, Reviews, Users, Settings, Ads, Notifications).
2. Converts the aggregated database objects into a single formatted JSON dump file.
3. Saves the dump file in `backend/backups/backup_[timestamp].json` .
4. Exposes REST controllers so admins can download, delete, or perform instant collection restorations from previous backups.